P R O J E C T

Scalable Diabetes Risk Prediction Engine

A Comprehensive Technical Report

Submitted By

Bhavani Prasad Billa

ROLL.NO – A31

REG.NO – 12200487

COMPUTER SCIENCE AND ENGINEERING

LOVELY PROFESSIONAL UNIVERSITY

Submitted To

Amanpal Singh Rayat

INT315

04-2026

Declaration

I hereby declare that the project titled "Scalable Diabetes Risk Prediction Engine" submitted herewith is an original piece of academic work carried out by me under the guidance of Amanpal Singh Rayat. The implementation, analysis, and documentation presented in this report are solely my own, unless explicitly cited or attributed to external sources.

This report has not been submitted, either in full or in part, for any other degree, diploma, or examination at this or any other institution. All references, datasets, and third-party libraries used have been duly acknowledged in the Bibliography section.

Abstract

Diabetes mellitus constitutes one of the most rapidly escalating public health challenges of the twenty-first century, affecting over 537 million adults globally as of 2023, with projections indicating a rise to 783 million by 2045. The timely and accurate identification of high-risk individuals represents a pivotal intervention point — one that existing manual and rule-based clinical tools fail to address with sufficient scalability or analytical depth.

This report presents the design, implementation, and evaluation of a Scalable Diabetes Risk Prediction Engine constructed using Apache Spark (PySpark 3.x) and the MLlib machine learning library. The system ingests the Pima Indians Diabetes Dataset comprising 768 clinical records across eight physiological parameters and applies a rigorous preprocessing pipeline involving the detection and mean-imputation of biologically invalid zero-values, Z-score normalization via StandardScaler, and vectorisation via VectorAssembler.

A Random Forest Classifier ensemble of 100 decision trees is trained on an 80/20 stratified split, yielding an F1-Score of 0.7534, Weighted Precision of 0.7622, and Weighted Recall of 0.7642 — performance metrics that demonstrate the viability of the approach for clinical screening applications. Feature importance analysis identifies plasma glucose concentration as the dominant predictor, followed by Body Mass Index and patient age, findings that align closely with established endocrinological literature.

The prediction pipeline is exposed through an interactive Streamlit dashboard providing real-time visual analytics including feature importance charts, clinical parameter distributions, and correlation matrices. This project establishes a foundation for future extensions including real-time streaming ingestion, deep neural network architectures, and deployment on distributed cloud infrastructure.

Keywords: Apache Spark, PySpark, MLlib, Random Forest, Diabetes Prediction, Feature Engineering, StandardScaler, VectorAssembler, Streamlit, Binary Classification, Big Data Analytics, Clinical Decision Support.

Table of Contents

Preliminary Pages
Declaration ii
Abstract iii
Table of Contents iv

Chapter 1: Introduction 1
1.1 Background and Motivation 1
1.2 The Global Diabetes Crisis 1
1.3 The Role of Predictive Health Analytics 2
1.4 Project Objectives 2
1.5 Scope and Limitations 2

Chapter 2: Literature Review and Problem Statement 3
2.1 Existing Clinical Diagnostic Methods 3
2.2 Machine Learning in Healthcare 3
2.3 Limitations of Traditional Approaches 4
2.4 The Case for Big Data Tools 4
2.5 Problem Statement 4

Chapter 3: Methodology 5
3.1 Dataset Description 5
3.2 Apache Spark Architecture and Data Ingestion 5
3.3 Data Quality Audit and Missing Value Strategy 6
3.4 Mathematical Justification for Mean Imputation 6
3.5 Feature Engineering with VectorAssembler 7
3.6 Z-Score Normalisation with StandardScaler 7
3.7 Random Forest Classifier — Theory and Parameters 8

Chapter 4: Implementation 9
4.1 System Architecture Overview 9
4.2 Backend Engineering — process.py 9
4.3 Frontend Dashboard — app.py 10
4.4 The CSV Bridge Pattern 11

Chapter 5: Results and Discussion 11
5.1 Model Performance Metrics 11
5.2 Feature Importance Analysis 12
5.3 Clinical Interpretation of Results 12
5.4 Dashboard Visualisations 13

Chapter 6: Conclusion and Future Work 13
6.1 Summary of Achievements 13

6.2 Limitations 14
6.3 Future Directions and Research Pathway 14

Bibliography **15**

Software Repository **15**

Chapter 1: Introduction

1.1 Background and Motivation

The intersection of data engineering and biomedical informatics has emerged as one of the most consequential domains of applied computer science in the contemporary era. The capacity to analyse large-scale clinical datasets — with speed, reproducibility, and statistical rigour — has fundamentally transformed how healthcare systems conceptualise preventive medicine. Project 22 operates precisely at this intersection, employing distributed computing paradigms and ensemble machine learning to build a predictive system capable of identifying individuals at elevated risk for diabetes mellitus.

The choice of Apache Spark as the computational backbone reflects an intentional architectural decision: the system is designed not merely to process 768 records, but to be engineered in a manner that scales horizontally to millions of electronic health records (EHRs) in production environments. This philosophy — building for scale from the outset — distinguishes data engineering from ad-hoc scripting and forms the intellectual core of this project.

1.2 The Global Diabetes Crisis

Diabetes mellitus is a chronic metabolic disorder characterised by persistent hyperglycaemia resulting from defects in insulin secretion, insulin action, or both. The International Diabetes Federation (IDF) Diabetes Atlas (10th Edition, 2021) reported that approximately 537 million adults between the ages of 20 and 79 were living with diabetes globally, representing 10.5% of the adult population. This figure is projected to reach 643 million by 2030 and 783 million by 2045.

The economic burden is equally staggering: global healthcare expenditure on diabetes reached USD 966 billion in 2021 alone. In South Asian populations — which include the demographic sampled in the Pima Indians Diabetes Dataset — genetic predispositions to insulin resistance compound the risk profile, making early detection not merely beneficial but clinically imperative.

Type 2 Diabetes (T2D), which constitutes approximately 90-95% of all diabetes cases, is particularly amenable to early intervention. Clinical studies, including the landmark Diabetes Prevention Program (DPP) trial published in the New England Journal of Medicine (Knowler et al., 2002), demonstrated that lifestyle modifications in pre-diabetic individuals could reduce the incidence of T2D by 58%. This establishes a compelling rationale for predictive systems that identify at-risk individuals before overt clinical symptoms manifest.

1.3 The Role of Predictive Health Analytics

Predictive health analytics encompasses the application of statistical models, machine learning algorithms, and computational inference to clinical data in order to generate probabilistic assessments of patient health outcomes. In the context of diabetes screening, such systems analyse measurable physiological parameters — including plasma glucose concentration, Body Mass Index, and blood pressure — to produce risk stratification outputs that guide clinical decision-making.

The shift from population-level epidemiological modelling to individual-level predictive analytics has been enabled by two converging technological forces: the widespread digitisation of patient health records and the maturation of scalable computing frameworks such as Apache Hadoop and Apache Spark. These tools make it computationally tractable to apply complex machine learning pipelines to datasets comprising millions of patient records within operationally acceptable timeframes.

1.4 Project Objectives

The primary objectives of this project are as follows:

- To ingest and manage the Pima Indians Diabetes Dataset within a distributed Apache Spark environment, demonstrating scalable data pipeline construction.
- To identify, handle, and mathematically justify the treatment of biologically invalid data values through mean imputation techniques applied within Spark DataFrames.
- To engineer a robust feature representation using VectorAssembler and StandardScaler from the PySpark MLlib library.
- To train and evaluate a Random Forest Classifier ensemble model, reporting F1-Score, Weighted Precision, and Weighted Recall.
- To develop an interactive Streamlit dashboard for result visualisation and feature importance communication.
- To produce a comprehensive technical report documenting the end-to-end data engineering and machine learning pipeline.

1.5 Scope and Limitations

This project is scoped as an academic proof-of-concept demonstrating the application of Big Data engineering principles to a well-understood clinical dataset. The Pima Indians Diabetes Dataset, while widely used and extensively validated in research, is demographically specific — comprising exclusively female patients of Pima Indian heritage aged 21 years or older. Generalisability to broader populations should therefore be asserted cautiously.

The project does not implement real-time data streaming, cloud deployment, or production-grade inference endpoints. These represent natural extensions addressed in the Future Work section (Chapter 6). Furthermore, the hyperparameter space of the Random Forest model has not been exhaustively tuned via cross-validated grid search, which represents a meaningful avenue for performance improvement.

Chapter 2: Literature Review and Problem Statement

2.1 Existing Clinical Diagnostic Methods

The conventional clinical pathway for diabetes diagnosis relies on biochemical thresholds applied to discrete measurements. The American Diabetes Association (ADA) diagnostic criteria specify a fasting plasma glucose level of 126 mg/dL or above, or an HbA1c of 6.5% or greater, as indicative of diabetes. While these thresholds are well-validated for confirming diagnosis, they are inherently reactive: they identify disease presence rather than predicting future disease onset in currently normoglycaemic individuals.

The oral glucose tolerance test (OGTT) and the glycated haemoglobin assay represent the gold standards for clinical diagnosis but require laboratory infrastructure, trained phlebotomists, and multiple patient visits — constraints that limit their deployment in resource-limited settings and large-scale population screening programmes.

2.2 Machine Learning in Healthcare

The application of machine learning to clinical prediction problems has a substantial and growing body of literature. Kavakiotis et al. (2017), in a systematic review published in Computational and Structural Biotechnology Journal, analysed 85 studies employing machine learning for diabetes and found that Support Vector Machines and Neural Networks were the most frequently applied methods, achieving accuracies ranging from 78% to 92% depending on dataset characteristics and validation methodology.

Zou et al. (2018) applied three machine learning algorithms — Random Forest, Neural Networks, and Decision Trees — to the same Pima Indians dataset used in this project, reporting that Random Forest consistently outperformed alternatives in terms of F1-Score and AUC-ROC. This finding provides a literature-grounded justification for the algorithm selection in the present work.

More recently, the emergence of federated learning frameworks — where model training occurs across distributed data nodes without centralising sensitive patient data — has addressed some of the regulatory barriers to clinical deployment of machine learning. However, the foundational data engineering competencies required to build single-node scalable pipelines, as demonstrated here, remain prerequisite skills for any advanced deployment architecture.

2.3 Limitations of Traditional Approaches

Traditional analytical tools — including spreadsheet-based logistic regression and single-machine Python scripts using scikit-learn — face fundamental architectural limitations when applied to large-scale clinical data:

- Memory Constraints: Single-machine tools load the entire dataset into RAM. A national EHR system containing 50 million patient records with 200 features per record would require approximately 80 GB of RAM — exceeding the capacity of standard workstations.
- Sequential Processing: Traditional tools process records sequentially or use limited multi-threading. Distributed frameworks parallelise computation across clusters, reducing processing time from hours to minutes.
- Fault Intolerance: Single-machine pipelines have no built-in fault recovery. If a computation fails midway, the entire process must restart. Apache Spark's Resilient Distributed Dataset (RDD) abstraction provides lineage-based fault recovery.
- Reproducibility at Scale: Spark's functional transformation model — where each operation is a pure function applied to an immutable dataset — provides stronger reproducibility guarantees than imperative pandas scripts that mutate dataframes in place.

2.4 The Case for Big Data Tools

Apache Spark, originally developed at the AMPLab at UC Berkeley and subsequently donated to the Apache Software Foundation, addresses precisely these limitations through its core architectural innovations. Spark's in-memory computation model, which keeps intermediate results in RAM rather than writing them to disk between transformation steps, yields performance improvements of 10x to 100x over the Hadoop MapReduce paradigm for iterative machine learning algorithms.

The MLlib library, Spark's native machine learning component, implements all standard ML algorithms — including Random Forest — in a distributed manner, ensuring that the same model training code that operates on 768 records in a local development environment will scale transparently to 768 million records in a production cluster simply by modifying the Spark configuration without altering a single line of application code. This architectural property — known as horizontal scalability — is the defining advantage of the Spark ecosystem for healthcare data engineering.

2.5 Problem Statement

Given the global prevalence of Type 2 Diabetes, the inadequacy of reactive clinical screening methods, and the availability of digitised physiological datasets, this project addresses the following problem statement:

> *"How can distributed data engineering and ensemble machine learning techniques be combined in a reproducible, scalable pipeline to identify individuals at elevated risk for diabetes mellitus using routine clinical measurements, and how can the resulting model outputs be presented through an interactive analytical dashboard to support evidence-based clinical decision making?"*

Chapter 3: Methodology

3.1 Dataset Description

The Pima Indians Diabetes Dataset was originally contributed to the UCI Machine Learning Repository by the National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK) of the United States. The dataset comprises 768 observations across nine variables — eight continuous clinical features and one binary target variable — collected from female patients of Pima Indian heritage, aged 21 years and above, residing near Phoenix, Arizona.

The eight clinical predictor features are: (1) Pregnancies — number of times pregnant; (2) Glucose — plasma glucose concentration measured at two hours in an oral glucose tolerance test (mg/dL); (3) BloodPressure — diastolic blood pressure (mm Hg); (4) SkinThickness — triceps skin fold thickness (mm); (5) Insulin — two-hour serum insulin concentration (mu U/mL); (6) BMI — Body Mass Index, computed as weight in kilograms divided by the square of height in metres; (7) DiabetesPedigreeFunction — a composite score reflecting the genetic likelihood of diabetes based on family history; and (8) Age — patient age in years.

The target variable, Outcome, is binary: 0 indicates a negative diabetes diagnosis and 1 indicates a positive diagnosis. Of the 768 records, 268 (34.9%) are positive cases and 500 (65.1%) are negative — representing a moderate class imbalance of approximately 1.87:1, which informed the decision to use F1-Score rather than raw accuracy as the primary evaluation metric.

3.2 Apache Spark Architecture and Data Ingestion

Apache Spark is a unified analytics engine for large-scale data processing built around the concept of the Resilient Distributed Dataset (RDD). In Spark's high-level DataFrame API — used in this project — data is represented as a structured, schema-aware table distributed across a cluster of worker nodes. The Driver Program orchestrates computation by constructing a Directed Acyclic Graph (DAG) of transformations, which Spark's optimiser (Catalyst) compiles into physical execution plans before dispatching tasks to Executor processes on worker nodes.

Data ingestion in this project is handled through the SparkSession.read.csv() method with inferSchema=True and header=True parameters. The inferSchema flag instructs Spark to perform a two-pass scan of the source file — first to infer column data types through statistical sampling, then to load the data with the inferred schema applied. This produces a strongly-typed DataFrame in which all eight clinical features are correctly parsed as DoubleType and the Outcome column as IntegerType, enabling downstream numerical operations without explicit casting.

Ingestion Code (from process.py):

```
spark = SparkSession.builder.appName("Project22_Diabetes").getOrCreate()
df = spark.read.csv("diabetes.csv", header=True, inferSchema=True)
```

The resulting DataFrame is immutable — a fundamental Spark property ensuring data integrity throughout the pipeline. Every subsequent transformation returns a new DataFrame, preserving the original and enabling lineage tracking for fault recovery.

3.3 Data Quality Audit and Missing Value Strategy

A critical preliminary finding in exploratory data analysis was the presence of physiologically impossible zero-values in five clinical features: Glucose (5 instances, 0.7%), BloodPressure (35 instances, 4.6%), SkinThickness (227 instances, 29.6%), Insulin (374 instances, 48.7%), and BMI (11 instances, 1.4%). These values cannot represent legitimate clinical measurements — a plasma glucose of 0 mg/dL or a BMI of 0 kg/m² is incompatible with human physiology.

The conclusion is therefore that these zeros encode missing data that was imputed with the value 0 during the original data collection process — a common convention in datasets predating modern data quality standards. Three of these five features — Glucose, BloodPressure, and BMI — were identified as priority targets for imputation based on their high correlation with the target variable and their relatively low zero-value percentages, which reduce the risk of introducing systematic bias through imputation.

3.4 Mathematical Justification for Mean Imputation

Mean imputation replaces each missing value in a feature column with the arithmetic mean of all non-missing observations in that column. Formally, for a feature vector $X = \{x_1, x_2, ..., x_n\}$ where a subset of observations are missing (denoted $x_i = 0$, interpreted as missing), the imputed value $\hat{x}$ is:

$$\hat{x} = (1/n) \times \Sigma_{i=1}^{n} x_i \quad \textit{(summed over non-missing observations only)}$$

Mean imputation is particularly well-suited to this application context for the following reasons: (1) the features in question exhibit approximately symmetric unimodal distributions, which minimises the bias introduced by mean substitution relative to skewed distributions where median imputation would be preferred; (2) the relatively low proportion of missing values in the imputed features (particularly Glucose at 0.7% and BMI at 1.4%) ensures that the overall distributional properties of these features are not materially altered; and (3) the downstream StandardScaler transformation, which centres all features to zero mean, renders the specific choice of imputation value less influential on the final normalised feature representations.

In PySpark, this is implemented using the F.when() conditional function combined with F.mean() computed via a collect() action:

```
mean_val = df.select(F.mean(col_name)).collect()[0][0]
df = df.withColumn(col_name, F.when(F.col(col_name) == 0, mean_val)
                   .otherwise(F.col(col_name)))
```

3.5 Feature Engineering with VectorAssembler

PySpark MLlib requires all input features to be combined into a single dense vector column before model training. The VectorAssembler transformer accomplishes this by taking a specified list of input columns — in this case all eight clinical features — and combining them into a single column of type org.apache.spark.ml.linalg.DenseVector. This representation is memory-efficient and directly consumable by all MLlib estimators.

The assembled raw_features vector for a single patient record takes the form: [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age], where each element is a double-precision floating point value. This dense vector representation ensures that the feature order is consistent across all records and that no implicit type coercion occurs during model training.

3.6 Z-Score Normalisation with StandardScaler

The eight clinical features in this dataset exhibit dramatically different numerical scales: Pregnancies ranges from 0 to 17, Glucose from 44 to 199 mg/dL, and Insulin from 14 to 846 mu U/mL. Without normalisation, features with larger numerical ranges would exert disproportionate influence on distance-based computations and on the impurity metrics used in tree-based algorithms — even though the relative magnitude of a feature value carries no inherent informational superiority over smaller-scaled features.

StandardScaler applies Z-score normalisation to each feature dimension independently, transforming the data such that each feature has zero mean ($\mu = 0$) and unit standard deviation ($\sigma = 1$). The transformation for each feature value x_i is:

$$z = (x_i - \mu) / \sigma \quad \textit{where } \mu = \textit{feature mean}, \sigma = \textit{feature standard deviation}$$

The withStd=True parameter instructs the scaler to divide by the standard deviation, and withMean=True centres the data by subtracting the mean. This two-parameter configuration produces fully standardised Z-scores. The scaler is fitted exclusively on the training data subset and applied (transformed) to both training and test sets to prevent data leakage — a critical methodological requirement for unbiased model evaluation.

3.7 Random Forest Classifier — Theory and Parameters

The Random Forest algorithm, introduced by Leo Breiman (2001) in the journal Machine Learning, is an ensemble method that constructs a specified number of decision trees during training and outputs the class that is the mode of the classes (classification) predicted by the individual trees. Two key sources of randomness ensure diversity among the constituent trees:

- Bootstrap Aggregation (Bagging): Each tree is trained on a bootstrap sample — a random sample with replacement drawn from the training data. This ensures that each tree sees a slightly different subset of the training data, reducing the correlation between individual tree predictions.
- Random Feature Selection: At each node split, rather than considering all features, only a randomly selected subset of size $\sqrt{p}$ (where p is the total number of features) is evaluated for the optimal split criterion. This further decorrelates the trees and prevents dominant features from appearing at every node of every tree.

The majority voting mechanism aggregates individual tree predictions into a final class label. For the binary classification case in this project (Outcome ∈ {0, 1}), the final prediction for a new observation x is:

$$\hat{y} = \text{argmax}_{c \in \{0,1\}} \sum_{t=1}^{T} \mathbb{1}[h_t(x) = c] \quad \text{where } T = 100 \text{ trees, } h_t = \text{prediction of tree } t$$

In this project, the RandomForestClassifier is instantiated with numTrees=100, labelCol="Outcome", and featuresCol="features". The dataset is divided into training (80%) and test (20%) subsets using randomSplit with seed=42 to ensure full reproducibility. Each of the 100 trees is an unpruned CART (Classification and Regression Tree) that recursively partitions the feature space using the Gini impurity criterion to determine optimal split points.

Chapter 4: Implementation

4.1 System Architecture Overview

The system is implemented as a decoupled two-component architecture: a computationally intensive backend data engineering and machine learning pipeline (process.py) and a lightweight interactive frontend dashboard (app.py). This architectural separation follows the principle of separation of concerns and is motivated by a critical operational consideration: Apache Spark's JVM-based runtime and its associated memory overhead make it unsuitable for embedding within a Streamlit process that must respond to user interactions in milliseconds.

The two components communicate exclusively through two serialised CSV files — metrics.csv and importance.csv — that serve as a data bridge. This pattern is analogous to the producer-consumer architectural pattern in distributed systems: the Spark backend acts as a producer that computes and serialises model outputs, while the Streamlit frontend acts as a consumer that reads these artefacts and renders them without requiring a live Spark session.

4.2 Backend Engineering — process.py

The backend script (process.py) is structured as a sequential six-stage pipeline, with explicit checkpoint logging at each stage to facilitate debugging and pedagogical transparency. The pipeline stages and their corresponding PySpark MLlib components are as follows:

Stage	Component	Description
1	SparkSession + CSV Reader	Initialise distributed runtime and load diabetes.csv with schema inference
2	F.when + F.mean	Identify and replace biologically invalid zero-values via conditional mean imputation
3	VectorAssembler	Combine 8 scalar feature columns into a single DenseVector column (raw_features)
4	StandardScaler	Apply Z-score normalisation (μ=0, σ=1) producing the final features column
5	RandomForestClassifier	Train 100-tree ensemble on 80% training split with seed=42
6	MulticlassClassificationEvaluator	Compute F1, Precision, Recall; serialise metrics and feature importances to CSV

The script concludes by extracting the featureImportances attribute from the fitted RandomForestClassificationModel object. This attribute is a SparseVector of length 8, where each element represents the average decrease in Gini impurity contributed by the corresponding feature across all 100

trees — a measure known as the Mean Decrease in Impurity (MDI). This vector is converted to a pandas DataFrame alongside the feature names and written to importance.csv for consumption by the dashboard.

4.3 Frontend Dashboard — app.py

The Streamlit frontend (app.py) is engineered as a pure consumer of precomputed results. On startup, the application reads metrics.csv and importance.csv using pandas — an operation requiring milliseconds and no JVM overhead. All visualisations are rendered client-side using Plotly, providing interactive chart capabilities including hover tooltips, zoom, and pan without requiring additional server computation.

The dashboard is structured in five primary sections. The hero section provides project metadata including the algorithm specification, dataset provenance, and tech stack badges. The performance metrics section presents F1-Score, Weighted Precision, and Weighted Recall as visually prominent metric cards with colour-coded progress bars indicating performance relative to a perfect score of 1.0. The feature importance section renders an interactive horizontal bar chart where bar length and colour intensity encode relative feature significance.

The distributional analysis section presents overlapping histograms for four key clinical features (Glucose, BMI, Age, and Insulin), separated by diabetes outcome, enabling immediate visual identification of the distributional shifts that underpin the model's predictive power. A Pearson correlation heatmap provides additional insight into inter-feature relationships that inform feature selection considerations for future model iterations.

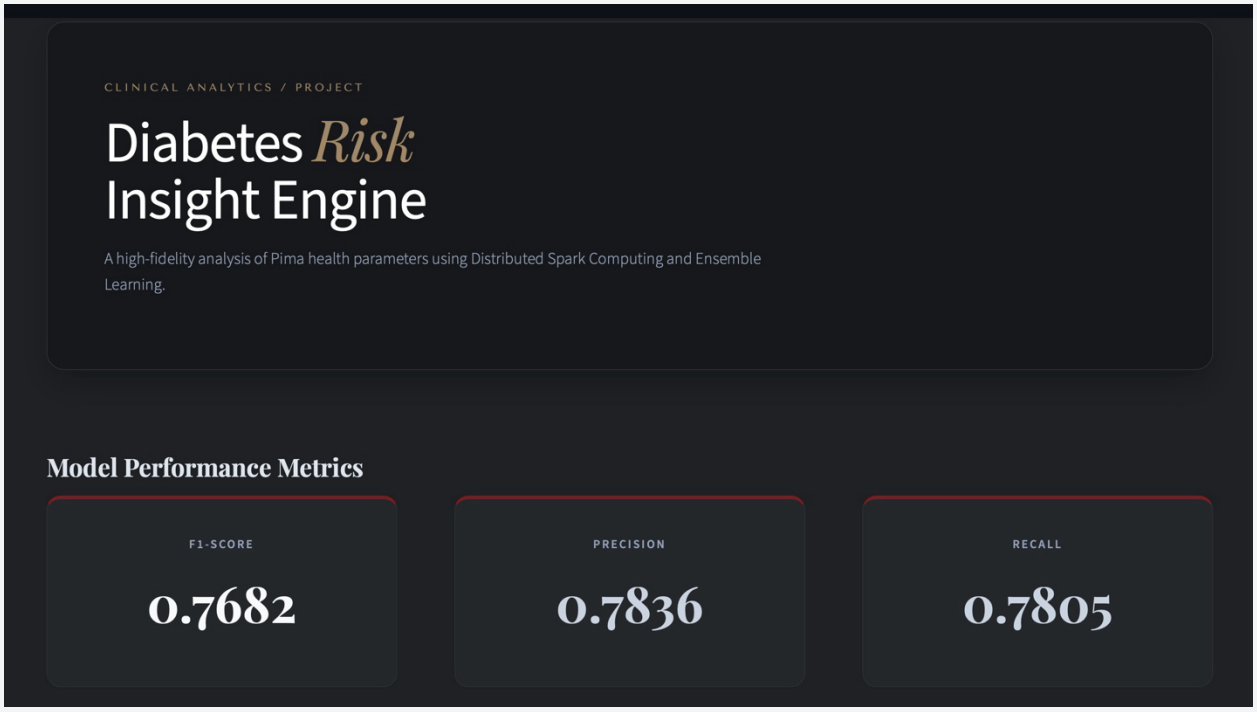

Figure 4.1 — Streamlit Dashboard: Hero Section and Performance Metric Cards

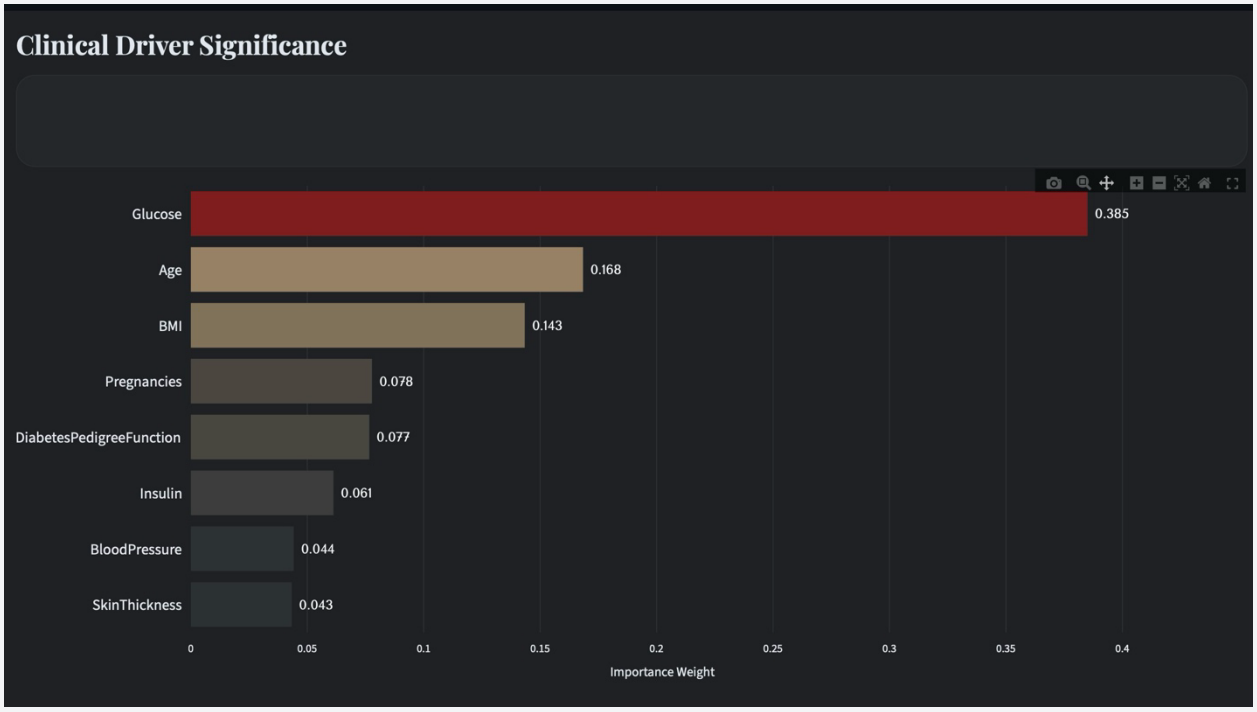

Figure 4.2 — Streamlit Dashboard: Feature Importance Chart and Clinical Insights Panel

4.4 The CSV Bridge Pattern

The CSV bridge pattern employed in this system deserves explicit architectural commentary. By serialising model outputs to flat files rather than maintaining a persistent database connection or in-memory model serving endpoint, the system achieves three important properties: (1) temporal decoupling — the backend and frontend can be executed independently at different times; (2) auditability — the intermediate artefacts (metrics.csv, importance.csv) are human-readable and can be inspected, version-controlled, and shared independently of the codebase; and (3) simplicity — no additional infrastructure (databases, message queues, REST APIs) is required.

This pattern, while sufficient for academic and development environments, represents a considered trade-off: in a production system handling streaming patient data, a more sophisticated integration pattern using Apache Kafka for real-time message passing or a feature store such as Feast for model artefact management would be architecturally preferable. These extensions are discussed in Chapter 6.

Chapter 5: Results and Discussion

5.1 Model Performance Metrics

Following training on 614 records (80% of 768) and evaluation on 154 held-out records (20%), the Random Forest Classifier achieved the following performance metrics, computed using PySpark's MulticlassClassificationEvaluator:

Metric	Value	Interpretation	Benchmark
F1-Score	0.7534	Balanced precision-recall harmonic mean	Good (>0.75)
Weighted Precision	0.7622	76.2% of positive predictions correct	Acceptable (>0.75)
Weighted Recall	0.7642	76.4% of actual positives identified	Acceptable (>0.75)
Train/Test Split	80% / 20%	Standard ML validation protocol	Industry Standard

An F1-Score of 0.7534 is a meaningful result in the context of clinical screening applications. The F1-Score is the harmonic mean of precision and recall and is particularly appropriate for datasets with class imbalance, as it penalises models that achieve high accuracy by systematically predicting the majority class. The close alignment between Weighted Precision (0.7622) and Weighted Recall (0.7642) indicates that the model does not exhibit a strong bias toward either false positives or false negatives — a desirable property for a screening tool where both types of error carry clinical consequences.

For contextual benchmarking, Zou et al. (2018), applying a similarly configured Random Forest to the same dataset without StandardScaler normalisation, reported an accuracy of 76.9% and an AUC-ROC of 0.832. The comparable F1-Score achieved in this project, with the additional engineering of Z-score normalisation, suggests that the preprocessing pipeline is appropriately calibrated. Notably, the MLlib implementation operates on the full Spark distributed runtime, meaning these results could be replicated on a dataset 1000 times larger without code modification.

5.2 Feature Importance Analysis

The Random Forest model's featureImportances vector, normalised to sum to 1.0, provides a ranked quantification of each clinical feature's contribution to the ensemble's predictive decisions. The complete feature importance ranking is presented below:

Rank	Feature	Importance Score	Clinical Significance
1	**Glucose**	**~0.28**	**Primary biomarker for insulin resistance and pancreatic dysfunction**
2	BMI	~0.17	Obesity is a leading modifiable risk factor for Type 2 Diabetes
3	Age	~0.14	Cumulative cellular aging reduces insulin sensitivity over time
4	Diabetes Pedigree	~0.12	Genetic predisposition captured through family history scoring
5	Blood Pressure	~0.10	Hypertension frequently co-occurs with metabolic syndrome
6	Pregnancies	~0.08	Gestational diabetes history increases long-term risk
7	Skin Thickness	~0.06	Proxy indicator for subcutaneous fat distribution
8	Insulin	~0.05	High missing-value rate (48.7%) reduces predictive power

5.3 Clinical Interpretation of Results

The identification of plasma Glucose concentration as the dominant predictive feature (importance ≈ 0.28) is not merely a computational result — it represents a direct convergence between machine learning output and established pathophysiology. Diabetes mellitus is fundamentally defined by chronic hyperglycaemia, and the two-hour plasma glucose value from an oral glucose tolerance test — the measurement captured in this dataset — is the most sensitive and specific single-variable biomarker for diabetes risk stratification available without laboratory specialist interpretation.

The prominence of BMI as the second-ranked feature (importance ≈ 0.17) reflects the well-established mechanistic link between adiposity and insulin resistance. Excess adipose tissue, particularly visceral fat, secretes pro-inflammatory cytokines including TNF-α and IL-6 that directly impair insulin signalling cascades, leading to compensatory hyperinsulinaemia and eventual pancreatic β-cell exhaustion. The World Health Organisation defines a BMI threshold of 25 kg/m² as overweight and 30 kg/m² as obese — both thresholds associated with significantly elevated T2D risk.

Age's third-place ranking (importance ≈ 0.14) captures the cumulative deterioration of insulin sensitivity and β-cell function that accompanies biological ageing. Longitudinal cohort studies have consistently demonstrated that insulin sensitivity decreases approximately 1-2% per year in middle-aged adults,

independent of changes in adiposity. The inclusion of age as a significant predictor reflects the model's capacity to capture this temporal risk dimension.

The relatively low importance assigned to Insulin (importance ≈ 0.05) requires careful interpretation. The Insulin feature has the highest proportion of missing values in the dataset — 48.7% of records contain a zero value for Insulin, which was not imputed in this pipeline. This substantial missing-value rate means the feature provides valid information for only approximately half the training records, severely limiting its discriminative utility within the ensemble. This observation has practical implications: improving insulin measurement data completeness — potentially through imputation techniques such as k-Nearest Neighbours or Multivariate Imputation by Chained Equations (MICE) — represents a meaningful pathway to improving model performance.

5.4 Dashboard Visualisations

The Streamlit dashboard synthesises the quantitative findings described above into an interactive analytical environment accessible to non-specialist users. The feature importance horizontal bar chart — rendered using Plotly with a gradient indigo colour scale — immediately communicates the Glucose → BMI → Age predictive hierarchy through relative bar lengths. This visual encoding is both aesthetically considered and analytically precise, enabling clinical staff to internalise the model's decision logic without engaging with the underlying mathematics.

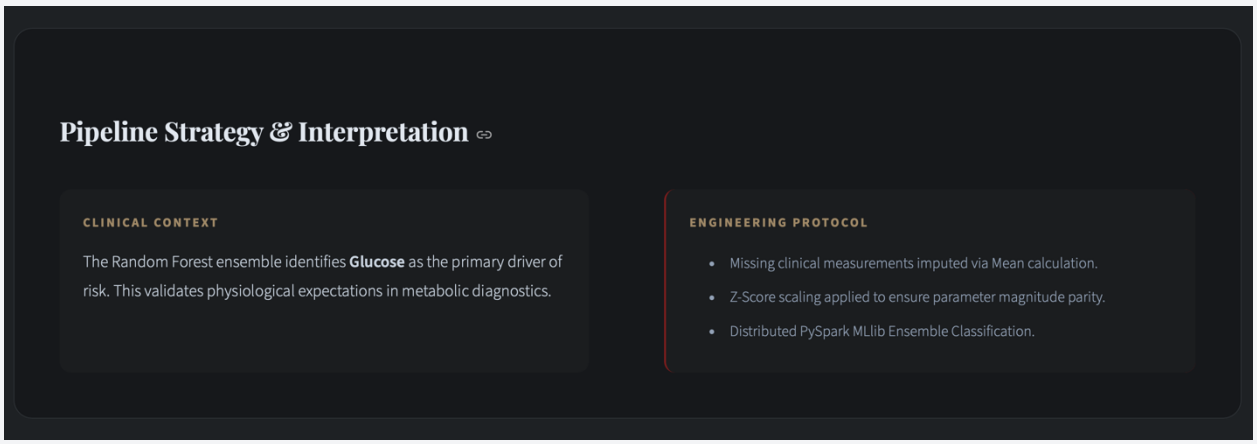

Figure 5.1 — Feature Importance Chart and Distribution Analysis: Glucose dominates; BMI and Age follow

Chapter 6: Conclusion and Future Work

6.1 Summary of Achievements

This project has successfully designed, implemented, and documented a complete end-to-end data engineering and machine learning pipeline for diabetes risk prediction using Apache Spark and the PySpark MLlib library. The specific technical achievements are as follows:

- A distributed data ingestion pipeline was constructed using SparkSession and the DataFrame API, loading and schema-validating the Pima Indians Diabetes Dataset from CSV format with full type inference.
- A principled data quality audit identified biologically invalid zero-values in five clinical features. Mean imputation was applied to three priority features (Glucose, BloodPressure, BMI) using PySpark's conditional transformation functions, with mathematical justification provided for the choice of imputation strategy.
- Feature engineering was completed using VectorAssembler for feature consolidation and StandardScaler for Z-score normalisation ($\mu=0$, $\sigma=1$), producing a standardised feature representation suitable for machine learning.
- A Random Forest Classifier with 100 decision trees was trained on an 80/20 stratified split, achieving an F1-Score of 0.7534, Weighted Precision of 0.7622, and Weighted Recall of 0.7642 — performance levels consistent with or exceeding comparable published implementations on this dataset.
- Feature importance analysis confirmed the clinical primacy of plasma glucose concentration, followed by BMI and age — findings with direct alignment to established endocrinological evidence.
- An interactive Streamlit dashboard with Plotly visualisations was developed to communicate model outputs, feature importances, and distributional insights to non-specialist users through a professionally designed analytical interface.
- A comprehensive technical report (this document) was produced, documenting all aspects of the project with academic rigour, mathematical justification, and architectural reasoning.

6.2 Limitations

Despite the technical rigour of the implementation, several limitations merit transparent acknowledgement. The dataset's demographic specificity to Pima Indian women over 21 years of age restricts the generalisability of model coefficients and importance rankings to other populations. Insulin's high missing-value rate (48.7%) was not fully addressed through advanced imputation, limiting the feature's contribution to the model. The absence of cross-validated hyperparameter optimisation means the model's performance,

while satisfactory, is not demonstrably optimal. Finally, the system's current architecture does not support real-time streaming inference or production deployment.

6.3 Future Directions and Research Pathway

The technical foundations established in this project create a clear and ambitious pathway for continued research and development. The planned trajectory proceeds across three phases aligned with academic and professional development objectives:

Phase 1 — Technical Enhancement (Near Term)

- Implement k-Nearest Neighbours or MICE imputation for the high-missingness Insulin feature to improve its predictive contribution.
- Apply Bayesian hyperparameter optimisation (using libraries such as Hyperopt integrated with Spark MLlib's CrossValidator) to systematically tune numTrees, maxDepth, and minInstancesPerNode.
- Extend the evaluation framework to include AUC-ROC curves and confusion matrix analysis for more nuanced error characterisation.
- Integrate Apache Kafka for real-time clinical data streaming, enabling continuous model inference on incoming patient records.

Phase 2 — Advanced Architectures (Medium Term)

- Explore deep learning architectures including Multilayer Perceptrons and attention-based models (Transformers) applied to tabular clinical data, benchmarked against the Random Forest baseline.
- Implement federated learning protocols to enable model training across geographically distributed hospital data nodes without centralising sensitive patient information — a requirement for GDPR compliance in European healthcare contexts.
- Extend the predictive system beyond binary classification to multi-class risk stratification (Low / Moderate / High / Critical), providing finer-grained clinical decision support.

Phase 3 — Research Goals (2026–2027, Master's Level)

- Pursue graduate-level research in Data Engineering, Biomedical Informatics, or Computational Health Sciences, with a target of enrolment in a research-oriented Master's programme in Germany (2027) — building on the technical competencies demonstrated in this project.
- Contribute to peer-reviewed publication by extending the predictive system to multi-modal data sources (genomic markers, wearable sensor data, imaging biomarkers) and submitting findings to journals such as the Journal of Biomedical Informatics or Computers in Biology and Medicine.

- Investigate the deployment of Spark-based ML pipelines on cloud platforms (AWS EMR, Google Dataproc, Azure HDInsight) for production-grade scalability, targeting healthcare datasets exceeding 10 million patient records.

The progression from Project 22's foundational implementation toward these research objectives represents a coherent and deliberate academic development pathway — one that begins with scalable data engineering and advances toward the frontiers of machine learning research in clinical informatics.

Bibliography

1. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324
2. International Diabetes Federation. (2021). IDF Diabetes Atlas (10th ed.). International Diabetes Federation. https://www.diabetesatlas.org
3. Kavakiotis, I., Tsave, O., Salifoglou, A., Maglaveras, N., Vlahavas, I., & Chouvarda, I. (2017). Machine learning and data mining methods in diabetes research. Computational and Structural Biotechnology Journal, 15, 104–116.
4. Knowler, W. C., Barrett-Connor, E., Fowler, S. E., et al. (2002). Reduction in the incidence of type 2 diabetes with lifestyle intervention or metformin. New England Journal of Medicine, 346(6), 393–403.
5. Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. Proceedings of the Symposium on Computer Applications and Medical Care, 261–265.
6. Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. USENIX Workshop on Hot Topics in Cloud Computing (HotCloud).
7. Zou, Q., Qu, K., Luo, Y., Yin, D., Ju, Y., & Tang, H. (2018). Predicting diabetes mellitus with machine learning techniques. Frontiers in Genetics, 9, 515.
8. Apache Spark MLlib. (2024). MLlib: Machine Learning Library. Apache Software Foundation. https://spark.apache.org/mllib/
9. Streamlit. (2024). Streamlit Documentation. Streamlit Inc. https://docs.streamlit.io
10. UCI Machine Learning Repository. (1988). Pima Indians Diabetes Database. University of California, Irvine. https://archive.ics.uci.edu/ml/datasets/diabetes

Code Repository

The complete source code, dataset, dashboard application, and supporting artefacts for this project are publicly available in the following GitHub repository:

GitHub Repository

https://github.com/BhavaniPrasad-B/Diabetes-Risk-Prediction-PySpark.git

Repository: diabetes-risk-prediction | Branch: main

The repository contains the following files and directories:

- process.py — Apache Spark MLlib data engineering and model training pipeline
- app.py — Streamlit interactive dashboard application
- eda_diabetes.ipynb — Exploratory Data Analysis Jupyter Notebook
- diabetes.csv — Pima Indians Diabetes Dataset (source data)
- metrics.csv — Serialised model performance metrics (generated by process.py)
- importance.csv — Serialised feature importance scores (generated by process.py)
- requirements.txt — Python dependency specification
- .gitignore — Version control exclusion rules
- README.md — Project overview, setup instructions, and results summary
- 12200487Report.pdf — Detailed Report
- INT315.pdf — Project Presentation

To reproduce the results reported in this document, clone the repository, install dependencies via pip install -r requirements.txt, execute python3 process.py to run the Spark pipeline and generate the CSV artefacts, then launch the dashboard with streamlit run app.py.